

1. The problem I am trying to solve is to build a cross-modal retrieval system that can match and compare real images taken by humans from the COCO and Flickr-30k datasets with their AI-generated images from Stable Diffusion and their associated captions or prompts. This is inspired by and an extension from the Kaggle competition [Stable Diffusion - Image to Prompts](#) where one has to build a model that inverts the text-to-image pipeline: Given an image generated by Stable Diffusion predict the original text prompt using the cosine similarity between the predicted embeddings and the true prompt embeddings. Concretely, I want to solve:

Given any image or text snippet, retrieve its most semantically similar counterparts across both real and AI-generated domains which is interesting because:

- It bridges real vs. AI domains. As generative models become ubiquitous, it's important to understand how well their latent spaces align with those of real images and human language.
 - It explores comprehensive cross-modal evaluation. Beyond than just matching captions to photos (the classic "text→image" retrieval), we hope to explore four retrieval modes: image↔image, text↔image, image→text, and "dataset→dataset". For example, if you upload an image, it returns the closest matches from COCO and Stable Diffusion for image-to-image. For text-to-image, one types a text and the system returns real photos that text describes and AI renders from Stable Diffusion. For image-to-text, we feed an image and should get text that describes it. For dataset-to-dataset example, Flickr-30k captions and ask which COCO captions are closest in meaning and vice versa.
 - Considering prompt quality assessment. By closing the loop (image→prompt→image), we can quantitatively measure how faithfully a generative model's output reflects the original content. This sheds light on prompt engineering efficacy.
 - A robust cross-modal retrieval pipeline has applications in content recommendation, image editing assistance, and automated captioning, and also provides benchmarks for future multimodal models.
2. The data that will be used to solve this problem involves images and captions. We will build and evaluate our cross-model retrieval system using 3 complementary text-image sources:
 - Stable Diffusion Image ↔ Prompt Pairs. The contents contain prompt-image pairs from Stable Diffusion. These are taken from the Kaggle competition [Stable Diffusion - Image to Prompts](#) then extract images and accompanying prompt text.
 - For MS COCO 2017 Captions, there are about 120,000 real-world images each with human-written captions. The images are taken from [here](#) and [here](#). We get the captions from [here](#).
 - For Flickr-30k, we get all of our data from a Kaggle competition [here](#) where we have 31,000 images and captions by humans.
 3. a. **Supervised vs. Unsupervised:** The core retrieval tasks (image↔image, text↔image, image→text, dataset→dataset) are handled unsupervised via nearest-neighbor search in a shared embedding space. The prompt-inversion step (predicting a prompt from its generated image) is a supervised regression problem.

b. **Classification or Regression:** retrieval uses no labels beyond the paired data as it's purely similarity-based. Inversion is regression, since we predict a continuous Contrastive Language-Image Pre-training (CLIP)-text embedding vector for each input image.

c. **What Are We Predicting?** Retrieval: we're not "predicting" labels in the traditional sense since we simply rank items by cosine similarity in embedding space. Prompt Inversion: we predict the CLIP text embedding of the original prompt given its generated image. Given an AI-generated image, try to recover (or approximate) the text prompt that produced it.

d. **Predictors (Features):** For all tasks, our "features" are the CLIP image embeddings for images and CLIP text embeddings for captions/prompts. For inversion, we may also experiment with additional visual features (e.g. CNN activations) or metadata (e.g. prompt length) as auxiliary inputs to the regression head.

e. **Traditional vs. Deep Learning:**

Deep Learning: Feature extraction using a pre-trained CLIP model to embed images and text. Prompt inversion: fine-tune a lightweight regression head on top of frozen CLIP image embeddings.

Traditional ML Baselines: TF-IDF + cosine: text→text retrieval baseline on captions and prompts. Linear/Ridge regression mapping image embeddings → text embeddings for inversion, to compare against the Multi-Layer Perceptron (MLP). This hybrid approach lets us leverage CLIP, while also training a small supervised model to invert the diffusion process and recover prompts from images.

4. We're building a search tool that understands both images and text. One will be able to use it as a web service with a clean API. The tool does:

- Find similar images: Give it a picture, and it'll find others like it.
- Search for images with words and vice-versa: Describe what one is looking for, and it'll pull up matching images.
- Discover related datasets: Find connections between entire collections of data.
- Reverse-engineer an image: Ever wonder what text prompt could create a specific AI-generated image? This tool will help figure that out.

To make it easy to try out, we're also creating an interactive web demo. We'll be able to upload any image or paste some text and instantly see the results. We'll even show newly generated images based on the text descriptions to see how well it's all working. We're focused on making a robust web service and a simple user interface for it. We're not building a mobile app.

5. a. **Processing power (CPU):** A modern 8-core/16-thread processor to support parallel data loading, preprocessing, embedding extraction, and API serving without bottlenecks.

b. **Memory:** At least 32 GB of system RAM so I can batch-process tens of thousands of embeddings and hold datasets (images + captions) comfortably in memory.

c. **Specialized hardware (GPU):** One NVIDIA GPU with ≥ 16 GB of VRAM (e.g. RTX 3090) to accelerate CLIP encoding and Stable Diffusion inference.